

ecalj note

<https://github.com/tkotani/ecalj>

August 9, 2025

This note becomes rather obsolete. I have to revise things step by step.

1 Product basis

This section is for the product basis to expand v and W within MTs.

```
<PRODUCT_BASIS>
  tolerance to remove products due to poor linear-independency
  0.100000D-04 ! =tolopt; larger gives smaller num. of product basis. See lbas and lbasC, which
  lcutmx(atom) = maximum l-cutoff for the product basis. =4 is required for atoms with valence
  4 3
  atom 1 nnvv nnc ! nnvv: num. of radial functions (valence) on the augmentation-
waves, nnc: num. for core.
  1 0 2 3
  1 1 2 2
  1 2 3 0
  1 3 2 0
  1 4 2 0
  2 0 2 1
  2 1 2 0
  2 2 2 0
  2 3 2 0
  2 4 2 0
  atom 1 n occ unocc ! Valence(1=yes,0=no)
  1 0 1 1 1 ! 4S_p -----
  1 0 2 1 0 ! 4S_d
  1 1 1 1 1 ! 4P_p
  1 1 2 0 0 ! 4P_d
  1 2 1 1 1 ! 4D_p
  1 2 2 0 0 ! 4D_d
  1 2 3 1 1 ! 3D_l
  1 3 1 0 1 ! 4f_p
  1 3 2 0 0 ! 4f_d
  1 4 1 0 0 ! 5g_p
  1 4 2 0 0 ! 5g_d
  2 0 1 1 1 ! 2S_p -----
  2 0 2 0 0 ! 2S_d
  2 1 1 1 1 ! 2P_p
  2 1 2 0 0 ! 2P_d
  2 2 1 1 1 ! 3d_p
  2 2 2 0 0 ! 3d_d
  2 3 1 0 1 ! 4f_p
  2 3 2 0 0 ! 4f_d
  2 4 1 0 0 ! 5g_p
  2 4 2 0 0 ! 5g_d
  atom 1 n occ unocc ForX0 ForSxc ! Core (1=yes, 0=no)
  1 0 1 0 0 0 0 ! 1S -----
  1 0 2 0 0 0 0 ! 2S
  1 0 3 0 0 0 0 ! 3S
  1 1 1 0 0 0 0 ! 2P
  1 1 2 0 0 0 0 ! 3P
  2 0 1 0 0 0 0 ! 1S -----
</PRODUCT_BASIS>
```

This section is read in the free format in fortran. So, e.g., 0.01 works as same as 0.10000D-01. The line order is important (you have to keep the order given by GWinput.tmp). Be careful

atom atom id—lmf may re-order it and pass it to gw code. Look into LMTO file (generated by **mkGWIN_lmf2**); which contains crystal structure information after such re-ordering by lmf. I used ! to make clear that things after ! are comments. But ! is not meaningful – just the expected numbers of data separated by blank(s) are read for each line from the beginning of lines.

- 0.100000D-02 ! =tolopt controls a number of Product basis to expand the Coulomb interaction within MTs. tolopt is a criterion to remove the poorly linear-independent product basis. Note that the product basis, which is to expand the Coulomb interaction, is different from the basis to expand eigenfunctions. In our experience, 0.100000D-02 (=0.001) is not so bad. If you like to reduce computational time use 0.01 or so, but a little dangerous in cases. With 0.0001, we can check stability on it.
(note: By supplying multiple numbers, we can specify tolopt atom by atom. Remember lmchk gives atom ID.)
- lcutmx(atom) is the l cutoff of product basis for atoms in the primitive cell (do lmchk for atom id). In the case of Oxygen, we can usually use lcutmx=2 (need check by the difference when you use lcutmx=2 or lcutmx=4). Then the computational time is reduced well.
- (dec2014:<PBASMAX> is not checked recently; see fpgw/main/hbasfp0.m.F and fpgw/gwsrsc/basnfp.F.) You can use <PBASMAX> section to override this setting. It is given as

```
<PBASMAX>
1  5 5 3 3
2  5 5 3 2 3
3  3 3 2 2 2
</PBASMAX>
```

The first number is for atom index (fixed), and other are product basis for each l channel.

- The integer numbers in 4th line lcutmx gives the maximum angular momentum l for the product basis for each atomic site. In our experience, lcutmx=4 is required when the semi-core (or valence) $3d$ electrons exist and we want to calculate the QP energies of them.
- Keep a block starting from " atom l nnvv nnc ..." as it originally generated in GWinput.tmp. It just shows that how many kinds of radial functions for cores and valence electrons for each atom and l . nnvv=2 in the case of ϕ and $\dot{\phi}$; nnvv=3 in the case to add the local orbital in addition.
- There are two blocks after the line "atom l n occ unocc :Valence(1=yes, 0=no)" and after "atom l n occ unocc ForX0 ForSxc ! Core (1=yes, 0=no)". These are used to choose atomic basis to construct the product basis. The product basis are generated from the products of two atomic basis.

GWinput.tmp generated by **mkGWIN_lmf2** contains labels on each orbitals as 4S_p, 4S_d, 4P_p... Here 4S_p is for ϕ_{4s} ; 4S_d for $\dot{\phi}_{4s}$; 3D_1 for ϕ_{3d}^{local} . Capital letter just after the

principle-quantum number means the orbital is used as ‘Head of MTO’; lowercase means just used only as the ‘tail of MTO’.

The switches for columns labeled as `occ` and `unocc.` take 0 (not included) or 1 (included). With the switch, we can construct two groups of orbitals, `occ` and `unocc.` In this sample `GWIN_V2` as for atom 1, $\{\phi_{4s}, \phi_{4s}, \phi_{4p}, \phi_{4d}, \phi_{3d}^{\text{local}}, \phi_{3s}^{\text{core}}, \phi_{3p}^{\text{core}}\}$ consist the group `occ`, and $\{\phi_{4s}, \phi_{4p}, \phi_{4d}, \phi_{3d}^{\text{local}}, \phi_{4f}\}$ consists the group `unocc.` So the any product of combinations $\{\phi_{4s}, \phi_{4s}, \phi_{4p}, \phi_{4d}, \phi_{3d}^{\text{local}}, \phi_{3s}^{\text{core}}, \phi_{3p}^{\text{core}}\} \times \{\phi_{4s}, \phi_{4p}, \phi_{4d}, \phi_{3d}^{\text{local}}, \phi_{4f}\}$ are included as for the basis of the product basis. As for atom 2, $\{\phi_{2s}, \phi_{2p}, \phi_{3d}\} \times \{\phi_{2s}, \phi_{2p}, \phi_{3d}, \phi_{4f}\}$ are included.

- Core section: (not worth to read, since we currently use no CORE2, A=B=C=0.)

Each line of the last section of `Product BASIS` forms

atom	1	n	occ	unocc	ForX0	ForSxc	:CoreState(1=yes, 0=no)
1	2	1	A	x	B	C	

At first you have to understand the concept of CORE1 and CORE2 in EQ.35 Ref.I. However, in our recent calculations, we do not use “CORE2” generally. So, in such a case, set A=B=C=0. And treat shallow cores (above Efermi−2Ry or so) as valence electron by “local orbital method” in `lmf`.

- Be careful. Current version is inconvenient... Need to repeat `mkGWIN_lmf2` to generate `GWinput` template when you add PZ (local orbital).

[(Note: you can skip here if you don’t use CORE2.)

Each of A,x,B,C takes 0 or 1. There are some possible combination of these switches;

1. If you take (A x B C)= (1 0 1 1), then the core is included in core2. In other words, this core is treated in the same manner of the valence electron.
2. If you take (A x B C)= (0 0 0 0), then the core is included in core1. The (exchange only) self-energy related to this core is included in `SEXcore`.
C is the key switch which determine whether it is included in core1 or core2. There could be another option.
3. If you take (A x B C)= (1 0 0 1). This core is in core2. But it is not included in the calculation of *D* and *W*. This core is only included for SEX and SEC calculations.

These three kinds of choices are reasonable ones but we can consider some another choice. In the following, we show how these switches (A,B,C) affect executions called from `gw_lmfh` (essentially as same as `gw_lmf`).

- `hbasfp0(mode 3)` :Product basis for exchange due to core.

We include the C=0 cores as a part of the product basis as if A=1 x=0.

- `hsfp0(mode 3)`: exchange mode for core.

Σ_x only due to the C=0 cores are calculated.

- **hbasfp0** (model): Product basis.
Only see the switch **A** and **x**. The product basis is generated from (occupied $\times$ unoccupied), where **A**=1 core is included as one of the occupied basis.
- **hsfp0** (mode 1): exchange mode.
Only see the switch **C**. Σ_x due to valence and due to **C**=1 cores are calculated.
- **hx0fp** (mode 1): $W - v$ calculation.
Only see the switch **B**. W is calculates using all the valence and **B**=1 cores.
- **hsfp0** (mode 2): correlation mode.
Only see the switch **C**. Σ_c due to valence and due to **C**=1 are calculated.

• After you perform **gw_lmfh** or anything, you find output files **lbas** by **hbasfp0** (model), and/or **lbas** by **hbasfp0** (mode3) for core. These contains important information about how many and how product basis are chosen. E.g. '**grep nbloch lbas**' shows how many product basis are used in the calculations.

<PRODUCT_BASIS>

```
tolerance to remove products due to poor linear-independency
0.100000D-02 ! =tolopt
```

When the product basis are made, we may have poorly linear independent basis. For example, one in the set $\{f_1, f_2, \dots, f_n\}$ would be almost give by a linear-combination of others. We need to make the linear-independent set. Therefore, after calculating the overlap matrix $\langle f_i | f_j \rangle$. We do diagonalization, then we remove eigenvectors corresponding to small eigenvalues than **tolopt**. See the **hbasfp0** command in **gwsc**

lcutmx(atom) = maximum l-cutoff for the product basis.

```
4 4 4 2 2 4 4
```

For $\phi_1 \times \phi_2$ case, $|l_1 - l_2| \leq l_{tot} \leq |l_1 + l_2|$. So '**lcutmx**' changes the maximum cutoff for the l_{tot} . The order is the same as the order of atoms in the **ctrl** file.

```
atom  l  nnvv  nnc !
1     0   3    3
1     1   3    2
1     2   2    1
```

'atom' means the atom number identified in the **ctrl** file.

'l' is the angular momentum quantum number.

'nnvv' is the number of radial functions (valence) on the augmentation-waves.

'nnc' is the number of radial functions for core.

The latter two ones, 'nnvv' and 'nnc', will be understood more clearly if you see the following ones.

atom	l	n	occ	unocc	! Valence(1=yes,0=no)
1	0	1	1	1	! 5S_p -----
1	0	2	0	0	! 5S_d
1	0	3	1	1	! 4S_l
1	1	1	1	1	! 5p_p
1	1	2	0	0	! 5p_d
1	1	3	1	1	! 4p_l

Above options are about the product basis set within MT (Valence).

‘atom’ and ‘l’ are explained above. ‘nnvv’ for ‘atom = 1 and l = 0’ was ‘3’ so this case we have 3 basis (‘n = 1, 2, 3’)

‘n’ is the degree of freedom of the radial function, ϕ . ‘n = 1’ means ϕ , ‘n = 2’ means $\dot{\phi}$, and ‘n = 3’ means kind of $\ddot{\phi}$, which the dot above the letter represents the differentiation with respect to the energy. So ‘n = 1 and 2’ is related to the linearization of the radial function and ‘n = 3’ is the local orbital which is restricted in MT. The local orbital can be modified changing ‘PZ’ in the **ctrl** file. Finally, the number of the basis set which is needed for expanding eigenfunctions is $(l+1)^2 \times n$.

‘occ’ and ‘unocc’ mean that we use only ones that checked as ‘1’, in other words we neglects ‘0’ cases for making product basis. Be careful for confusion with name ‘occ’ and ‘unocc’. These don’t mean that occupied or unocc. When making product basis, $M = \phi_1 \times \phi_2$, ‘occ’ corresponds to ϕ_1 and ‘unocc’ to ϕ_2 . For example,

atom	l	n	occ	unocc	! Valence(1=yes,0=no)
1	0	1	1	1	! 5S_p -----
2	3	1	0	1	! 4f_p

If the options are like the above, the product basis will be consists of $(\phi_1 = \phi_{atom=1,l=0}) \times (\phi_2 = \phi_{atom=1,l=0})$, $(\phi_{atom=1,l=0} \times \phi_{atom=2,l=3})$. As you can see, $(\phi_1 = \phi_{atom=2,l=3})$ is skipped.

In the **ctrl** file, ‘EH’ controls the l part. As for ‘EH’, (s, p, d, f) are used but **GWinput** file uses (s, p, d, f, g). ‘EH’ : HEAD part. ‘GWinput’ : contains TAIL part... need more explanation.

atom	l	n	occ	unocc	ForX0	ForSxc	! Core (1=yes, 0=no)
1	0	1	0	0	0	0	! 1S -----
1	0	2	0	0	0	0	! 2S
1	0	3	0	0	0	0	! 3S

Above options are about the product basis set within MT (Core).

‘nnc’ for ‘atom = 1 and l = 0’ was ‘3’ so this case we have 3 basis (‘n = 1, 2, 3’)

Finally, for the convergence check, we can modify the following three things, (i) tolerance, (ii) lcutmx, and (iii) occ and unoccu.

2 Wannier function

We can generate Wannier functions (maximally localized Wannier Functions or similar) by a script `genMLWF`. It automatically perform cRPA calculation sucessively. (If it is not necessary, insert 'exit' in `genMLWF`, after it performs `lmaxloc2`).

Try to run examples in `ecalj/MATERIALS/Sample_MLWF/`. Read README in it. To run the script `genMLWF`, we need to get `GWinput` by editing `GWinput.tmp`. (`mkGWin_lmf2` contains default Wannier section). In addition, we have some settings (energy windows and so on). This is the example of the initial conditions for Cu case. 5 is the number of Wannier function. The most left one means ϕ index and the right one of it is $\dot{\phi}$ index. They are written in the `@MNLA_CPHI` file.

Then we can run `genMLWF`. After it finished, we can analyze it results. (if you don't need Wannier function plot, You can skip a line of `wanplot` in `genMLWF`. Then we don't need to set `vis_wan_*` options.)

2.1 lwmatK1 and lwmatK2

If you input the following command

```
>grep Wan lwmatK*
```

You will get the following results. (This case : Cu cases)

lwmatK1:	Wannier	1	1	24.644475	0.000000 eV	
lwmatK1:	Wannier	1	2	24.644576	0.000000 eV	
lwmatK1:	Wannier	1	3	25.471361	0.000000 eV	
lwmatK1:	Wannier	1	4	24.644575	0.000000 eV	
lwmatK1:	Wannier	1	5	25.470946	0.000000 eV	
lwmatK2:	Wannier	1	1	0.000000 eV	-21.263759	-0.000000 eV
lwmatK2:	Wannier	1	2	0.000000 eV	-21.263839	0.000000 eV
lwmatK2:	Wannier	1	3	0.000000 eV	-21.931033	-0.000000 eV
lwmatK2:	Wannier	1	4	0.000000 eV	-21.263839	-0.000000 eV
lwmatK2:	Wannier	1	5	0.000000 eV	-21.930702	-0.000000 eV

Wanneir Branch now under developing (imported from T.Miyake's Wannier and H.Kino's).

A. make at `ecalj/fpgw/Wannier/` directory, and do make, and make install.

(need to check Makefile first). You first have to install `fpgw/exec/` in advance.

B. Samples are at these directories.

`MATERIALS/CuMLWFs` (small samples),

`MATERIALS/CuMLWF/`

`MATERIALS/CuMLWFs/`

`MATERIALS/FeMLWF/`

`MATERIALS/NiOMLWF/`

MATERIALS/SrVO3MLWF/

C. With GWinput and ctrl.*, run

```
>genMLWF
```

at these directories.

In GWinput, we supply settings to generate Wannier functions. (Sorry, not documented yet..)

D. After genMLWF, do

```
>grep Wan lwmatK*
```

then compare these with Result.grepWanlwmatK

These are onsite effective interactions (diagonal part only shown).

*.xsf are for plotting the Maximally localized Wannier functions.

Anyway, documentation on Wannier is on the way half.

Time consuming part (and also the advantage) is for effective interaction in RPA.

Look into the shell script genMLWF; you can skip last part if you don't need the effective interaction.

3 How to set local orbitals

As we stated, do "lmfa |grep conf" to check used MTO basis.

We have to set SPEC_ATOM_PZ=?,?,?

(they ordered as PZ=s,p,d,f,...) to set local orbitals.

lmv7 (originally due to ASA in Stuttgart) uses a special terminology

"continuous principle quantum number for each l", which is just

related to the logarithmic derivative of radial functions at MT

boundary. It is defined as

$P = \text{principleQuantumNumber} + 0.5 - 1/\pi \cdot \text{atan}(r \cdot 1/\phi \cdot d\phi/dr)$,

where ϕ is the radial function for each l. For example,

$P = n.5$ for $l=0$ of free electron (flat potential) because $\phi=r^0$,

$P = n.25$ for $l=1$ because $\phi=r^1$;

$P = n.147584$ for $l=2$ because $\phi=r^2$; $P = n.102416$ $n.077979$ for $l=3,4$.

(Integer part can be changed). See Logarithmic Derivative Parameters in

<http://titus.phy.qub.ac.uk/packages/LMTO/lmto.html#section2>

Its fractional part $0.5 - \text{atan}(1/\phi \cdot d\phi/dr)$ is closer to unity for

core like orbital, but closer to zero for extended orbitals.

Examples of choice:

Ga p: in this case, choice 0 or choice 2 is recommended.

We usually use l0 for semi-core, or virtually unoccupied level.

```

(0)no lo (4p as valence is default treatment without lo.)
    3p core, 4p valence, no lo: default.
    Then we have choice that lo is set to be for 3p,4p,5p.
(1)3p lo ---> 4p val (when 3p is treated as valence)
    3d semi core, 4d valence
    Set PZ=0,3.9
    (P is not required to set. *.9 for core like state. It is just an initial condition.)
(2)5p lo ---> 4p val (PZ>P)
    Set PZ=0,5.5
    5.5 is just simply given by a guess (no method have yet
implemented for
    If 5.2 or something, it may fails
    because of poorness in linear-dependency. We may need to observe
    results should not change so much on the value of PZ.

(3xxx)4p lo ---> 5p val (we don't use this usually. this is for test purpose)
    4p lo, 5p valence
    Set PZ=0,4.5 P=0,5.5 (In this case, set P= simultaneously).
    (NOTE: zero for s channel is to use defalut numbers for s)

Ga d: (in this case, choice 0 or choice 1 is recommended).
(0)no lo (3d core, 4d valecne, no lo: default.)
    Then we have choice that lo is set to be for 3d,4d,5d.
(1) 3d lo ---> 4d val (when 3d is treated as valence)
    Set PZ=0,0,3.9 (P is not required to set)
(2) 5d lo ---> 4d val (PZ>P)
    Set PZ=0,0,5.5
(3xxx) 4d lo ---> 5d val (this is for test purpose)
    Set PZ=0,0,5.5 P=0,0,4.5
    (NOTE: zero for s,p are to use defalut numbers )

% If you like to read from atm.ga file instead of rst file(if exist).
% You have to do lmf --rs=1,1,0,0,1, for example. See lmf --help
% Becase rst file keeps the setting of MTO, thus change in ctrl is not
% reflected without the above option to lmf.
=====

```

4 Linear response calculations

With these scripts for linear response calculations, **eps***, we can calculate **q**-dependent dielectric function $\epsilon(\omega, \mathbf{q})$ (and v , W) (and χ for spin fluctuation). But (because of numerical reason), we can not use $\mathbf{q} = 0$ limit. (if $|\mathbf{q}|$ is too small, we have numerical problem, zero divided by zero, because we have not implemented the version to use $\mathbf{q} = 0$.)

- **eps_lmfh**

Dielectric function epsilon with local field correction. Expensive calculation (we may need to reduce number of wing parts in future...).

- **epsPP_lmfh**

epsilon without local field correction. $1 - \langle e^{i\mathbf{qr}} | v | e^{i\mathbf{qr}} \rangle \langle e^{i\mathbf{qr}} | (\chi^0) | e^{i\mathbf{qr}} \rangle$

- **epsPP_lmfh_chipm**

For spin susceptibility. This essentially calculate non-interacting spin susceptibility. Then it is used for the calculation of full spin susceptibility with **util/calj_*.F** programs (small quick programs). See spin wave paper. See spin susceptibility section Sec.??.

- (not maintained now; we will recover this) **eps_lmfh_chipm**

This gives full non-interacting spin susceptibility. Testing. We have to determine U (Stoner I) for the determination of full spin susceptibility. TDLDA? or so?

- (This is old mode --- not maintained) **epsPP_lmfh_chipm_q**

For spin susceptibility, spin susceptibility $\langle e^{i\mathbf{qr}} | \chi(q, \omega) | e^{i\mathbf{qr}} \rangle$ In this script, You have to assign that $\text{isp}=1$ is majority, $\text{isp}=2$ is minority. This is with long wave approximation.

-
- We use the histogram method (the Hilbert transformation method); we first calculate its imaginary parts with the tetrahedron technique for dielectric functions. Then we get its real part by the Hilbert transformation.

You need to choose `HisBin_dw, HisBin_ratio`. The width of histogram bins are getting larger when omega gets larger. `dw` is the size of histogram-bin width at $\omega=0$. At $\omega=\omega_c$, its width gets twiced.

To plot dielectric function with reasonable resolution, it might be better to set `dw` 0.001 and `omg_c` 0.1 for example. You may have to choose small enough omega for spin wave mode as 0.001 Ry (Or smaller). `omg_c` is given like 0.05 Ry or so. But sometimes it can be like 1Ry.

- **epsPP_lmfh** only calculation an matrix element of dielectric function for $\exp(i\mathbf{qr})$. Thus very faster than **eps_lmfh** mode. It uses a a special product basis set for cases without inversion (problem is in how to expand $\exp(i\mathbf{qr})$ in the MPB; the product basis is not from phi and phidot, but from spherical Bessel functions).

In *_lmfh_* modes(I now use little for *_lmf_* modes), you can use small enough delta. Use small enough delta ($=-1e-8$ a.u.) for spin wave modes (also you can use it for dielectric function and GW). This is necessary because pole is too smeared if you use larger delta.

4.1 eps_lmfh, epsPP_lmfh: the dielectric functions

You can invoke the script, e.g. as "eps_lmfh si".

Specify $\mathbf{q}$ point in <QforEPS> or so. Mesh for ω is specified by `dw, omg_c`.

The obtained data are in EPS*.dat and EPS*.nlfc.dat. EPS*.nlfc.dat contains the result without local-field correction EPS*.dat contains the result with local-field correction (this is generated only for eps_lmfh. Both of them contains

$\mathbf{q}(1:3), \omega, \text{Re}(\epsilon), \text{Im}(\epsilon), \text{Re}(1/\epsilon), \text{Im}(1/\epsilon)$

in each line.

4.2 epsPP_lmfh: the dielectric function(No LFC— faster)

You can calculate ϵ without LFC by epsPP_lmfh. It is very faster than eps_lmfh.

To calculate $\epsilon(\mathbf{q}, \omega)$ without LFC accurately, the best basis set for the expansion of the Coulomb matrix within MT is apparently not the product basis, but the Bessel functions corresponding to the plane waves $\exp(i\mathbf{q}\mathbf{r})$. We use such a basis in this mode. However, our experience shows that the changes are little even with the usual product basis (we don't describe this here).

4.3 How to calculate correct dielectric function?

(this subsection is essentially OK... but need to clean it up. dec2014)

There are problems to calculate correct epsilon.

At first, we talk about epsPP_lmfh, which is No LFC. Main problem are

1. Convergence for number of k point(specified by n1n2n3).

Roughly speaking, 20x20x20 is required for not-so-bad results for Fe and Ni.

It is better to do 30x30x30 to see convergence check.

However, in the case of ZB-MnAs (maybe because of simple structure around Ef), it requires less q points.

figs are for GaAs.

fig001: n1n2n3 convergence for Chi_RegQbz = on case.

fig002: n1n2n3 convergence for Chi_RegQbz = off case.

(Chi_RegQbz is explained in General section in this manual).

As you see, k points convergence looks a little better in `Chi_RegQbz=off` (mesh not including γ). However a little problem is that its threshold around 0.5eV is too high and slowly changing.

fig003: Alouanis'(from Arnaud) vs. ```Chi_RegQbz = on''` vs. ```Chi_RegQbz = off''`
 As you see, the threshold of the Red line (20x20x20 `Chi_RegQbz=on`) and Alouani's are almost the same, but the red line is too oscillating at the low energy part. On the other hand, ```Chi_RegQbz = off''` in Green broken line is not so satisfactory at the low energy part.

fig.gas_eps_kconf.pdf shows the convergence behavior of epsilon for

2. $q \rightarrow 0$ convergence (this is related to whether `Chi_RegQbz=on` or `off`).

If you use very small q like $q=0.001$ in GaAs, it can cause a problem.

Use $q=0.01$ or larger (maybe $q=0.02$ or more is safer).

Very small q can give numerical error for high-energy region.

In fig004, we show the high energy tail part of $\text{Im } \epsilon(\omega)$ for GaAs case. At $q=0.01$ (this means $q = 2\pi/\text{alat} * (0.001)$), the imaginary part is a little too large. Less than 80eV, $q=0.02$ gives good results when compared with other high q results, though it still has noise above 80eV.

In fig005, I showed the same results compared with Alouani's (his is up to 40eV). Both gives rather good agreements. As you see, $q=0.06$ or above might be necessary to get reasonable convergence for high energy part above 40eV.

We have to be careful for this pooriness in high energy part--- it may effect low-energy $\text{Re } \epsilon$ through KK relation. However this can be very small enough.

In fig.gas_eps_qconv.jpg, we checked the convergence of $\epsilon(\omega=0, q)$ for $q \rightarrow 0$. As you see, it gives convergence, however, $q=0.01$ is a little out of curve---this should be because of the pooriness in the high energy part. so $q=0.02$ or $q=0.03$ is safer, and you can get ϵ within 1 percent accuracy.

3. Including Core for dielectric constant is dangerous.

It can cause very poor results if you include core part in `GWinput`.

You need to include core just as valence (with local orbital).

In fig008, we showed core effects. It starts from $\approx 16\text{eV}$

(this is core to conduction transition).
 fig007 showd the check about the q point dependence---even with large q,
 it would not change.
 These shows that the core excitation can have larger energy range.
 This is in contrast to the valence case
 (then the most of excitaion is limited to less than 10eV).
 We have to be careful for such high-energy exciation... The LMT0 basis might
 be not so good for high energy.

4. basis set.

Use QpGcut_psi \approx 3.0 a.u. or so (as same as GW calculation).
 In the case of epsPP* mode,
 QpGcut_cou can be very small--- In our codes now,
 ngc>=1 should be for all q vector shown in lqg4gw02 (output of echo 2|qg4gw).
 [In principle, it should be only for the q vector for which we calculate epsilon.
 But there is a technical poorness in our code---
 (maybe) a problem here; the plane-wave part of the eigenfunction generated
 in lmf2gw is not correctly passed to lmf2gw when ngc=0].

-- eps_lmfh: including LFC -----

To include eps with LFC, do eps_lmfh.

But lcutmx=2 seems to be good enough to get 0.5 percent error (maybe better than this).

Test it 10x10x10 or so. (I need to repeat if necessary).

Further you can use smaller QpGcut_cou like 2.2 or so,

with rather smaller product basis (up to p timed d, not including f).

Note: epsPP_lmfh is designed to use good basis to calculate eps
 without LFC. This is usually in agreement with what you obtained by eps_lmfh;
 however it can give slight difference when you use small product basis.

---Summary -----

So in conclusion, I think a best way to do is

1. set q=0.02 [q=2pi/alat(0 0 0.02)] or so for GaAs case.

If you want to check, do q=0.03 and q=0.06 also.

``Chi_RegQbz = off'' is better for matrials like GaAs with direct gap.

2. You can use small QpGcut_cou but all ngc should be one or more.

3. As for the Product basis setting in epsPP* scripts, only

lcutmx and tolerance (this can be like 0.001 or so) are relevant.to determine eps(omega=0, c

E.g. set lcutmx=4 or so.

5. To get eps with LFC, set QpGcut_cut as xxx, and set lcutmx=2 where

4. Do nk=20 18 16 and take interpolariion

(occupied sp) \timex (unoccupied spd) are included.

But correct EPS*.nolfc.d is rather from epsPP_lmfh script.

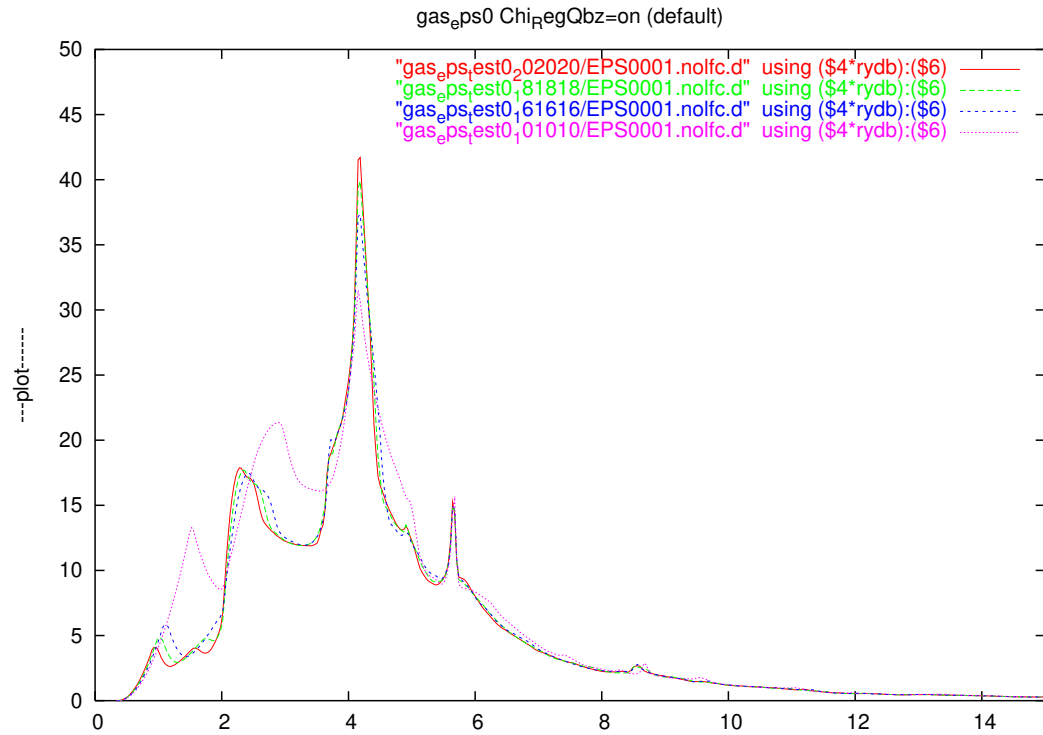


fig001

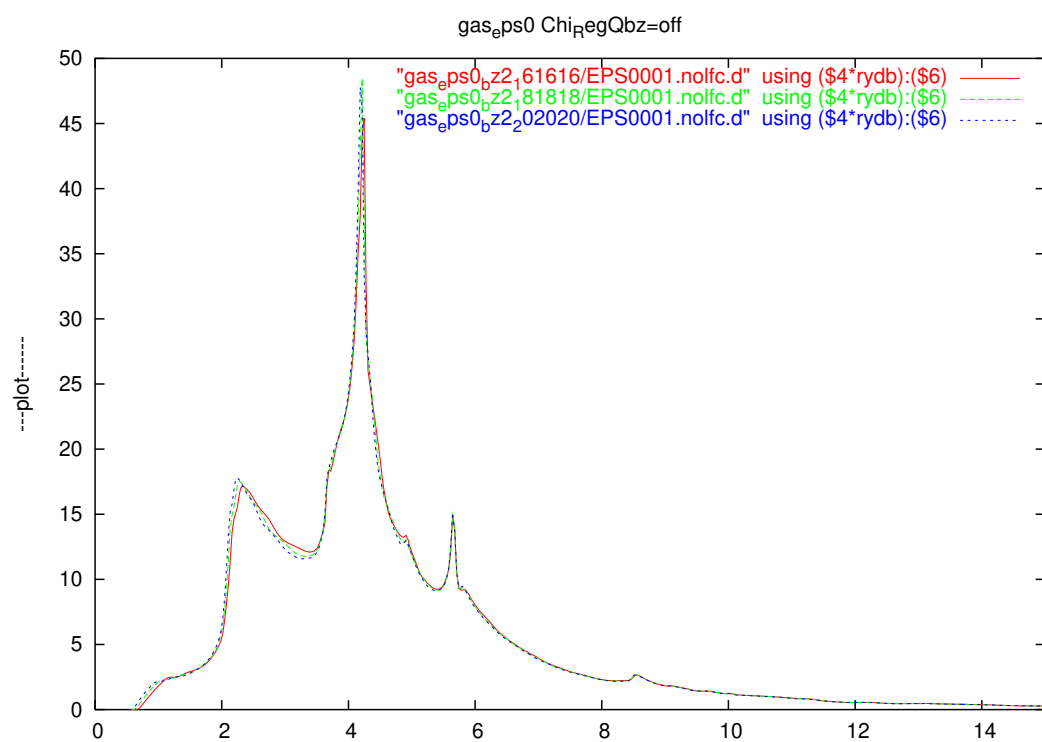


fig002

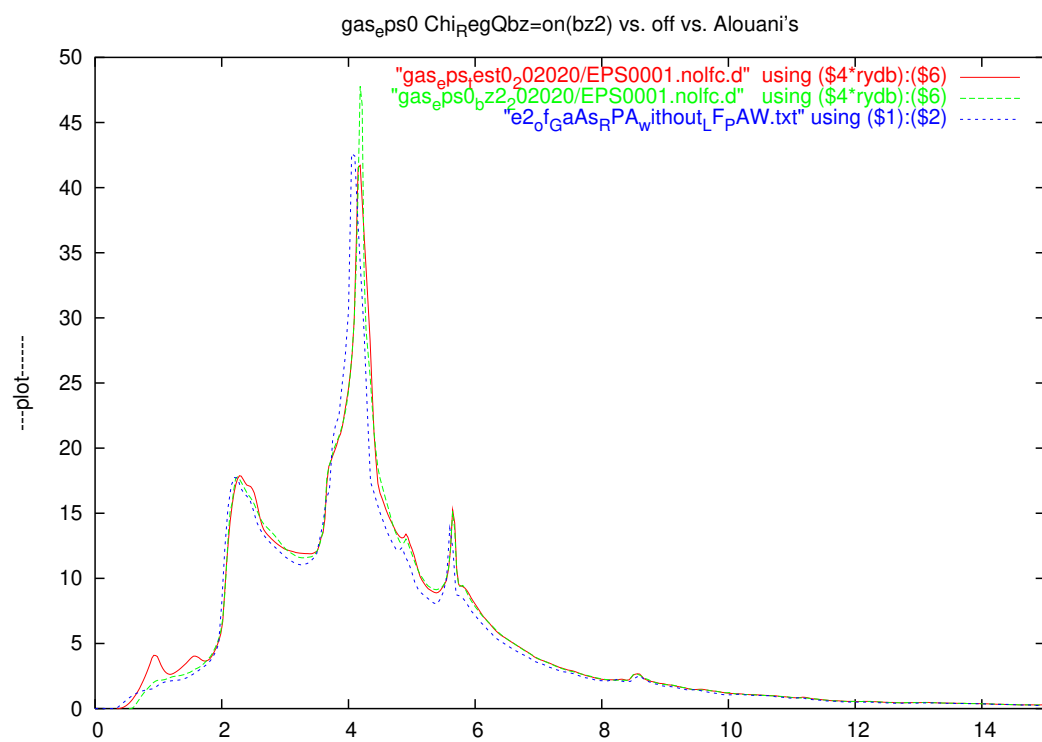


fig003

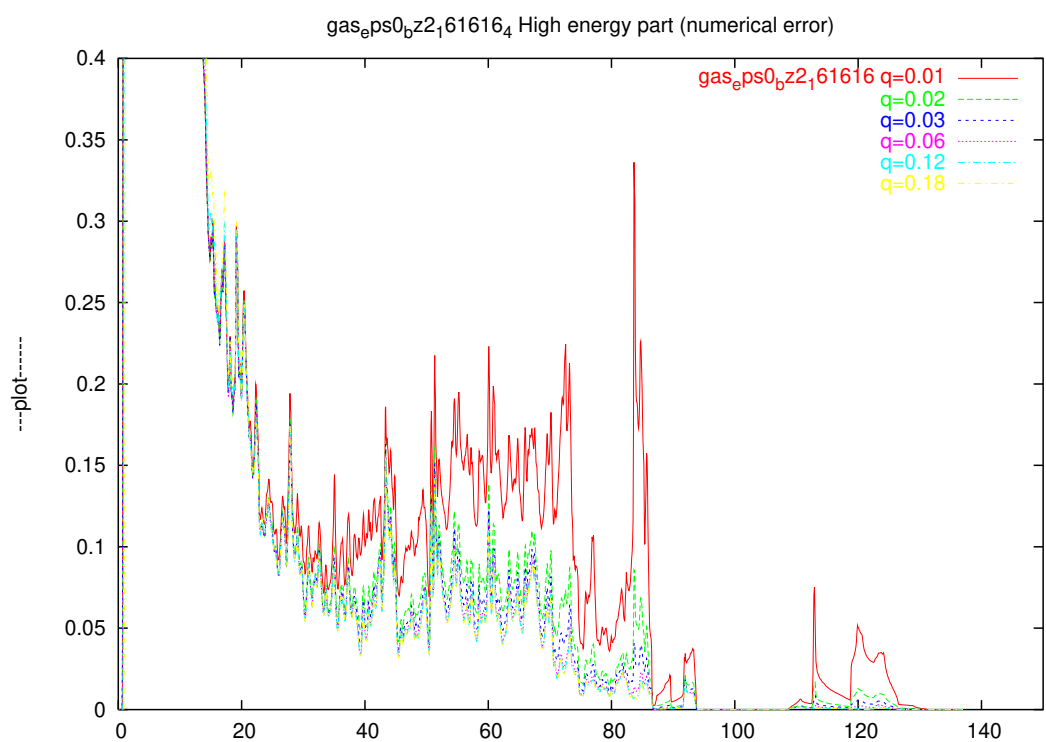


fig004

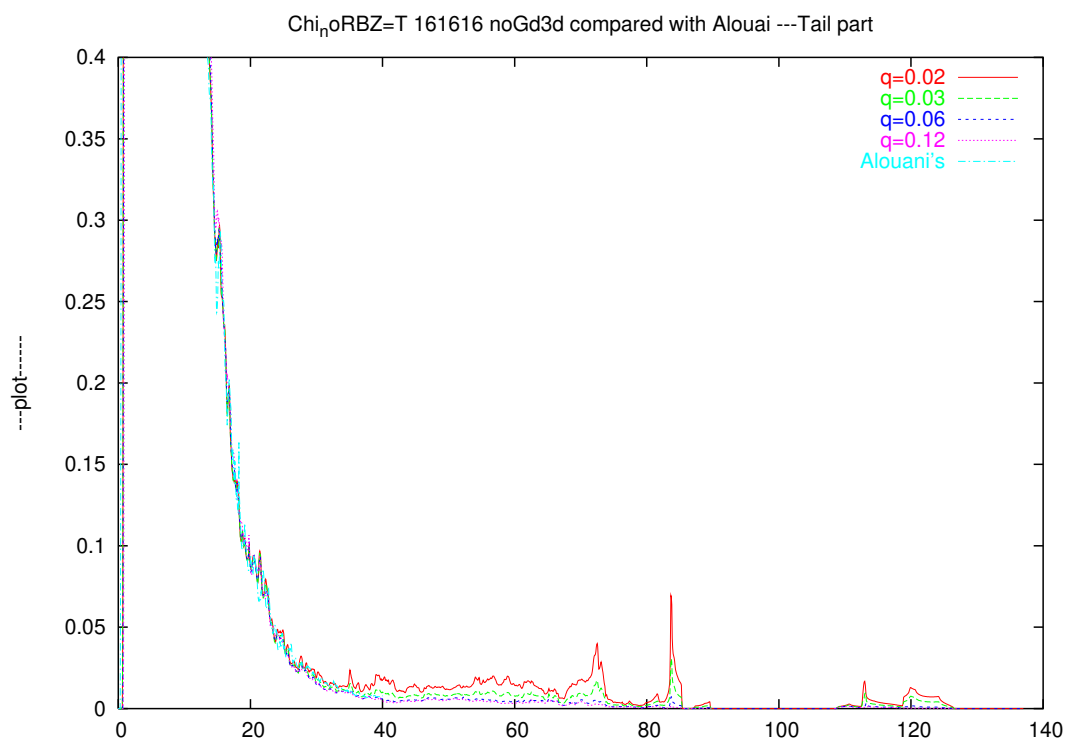


fig005

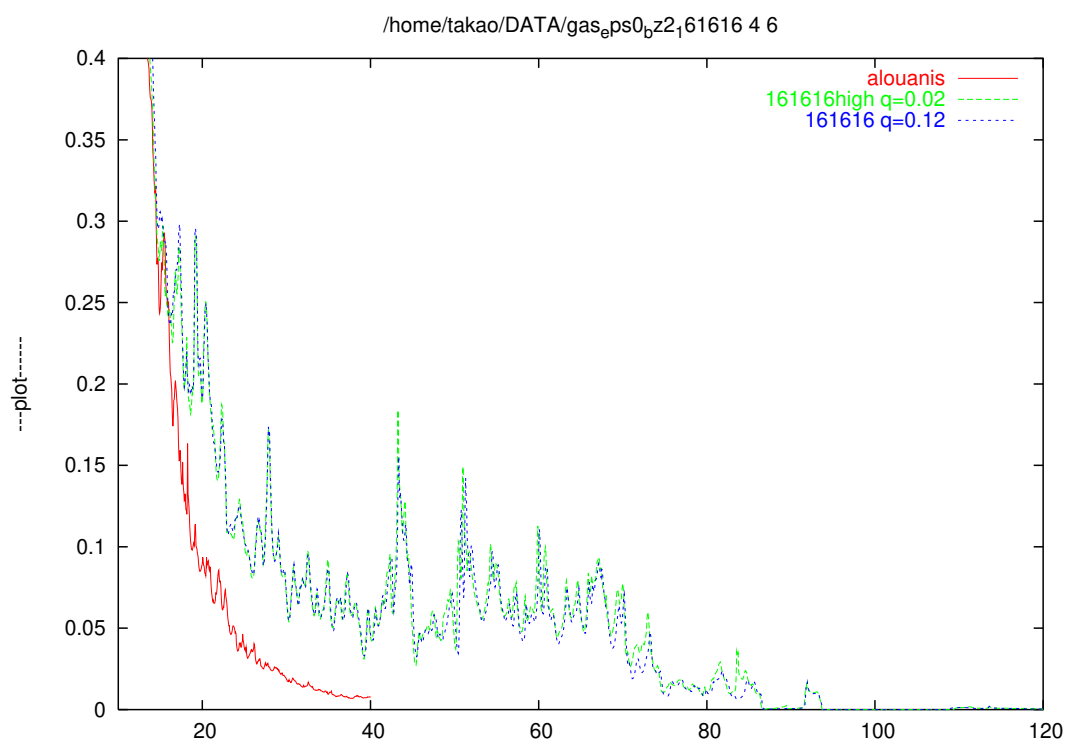


fig007

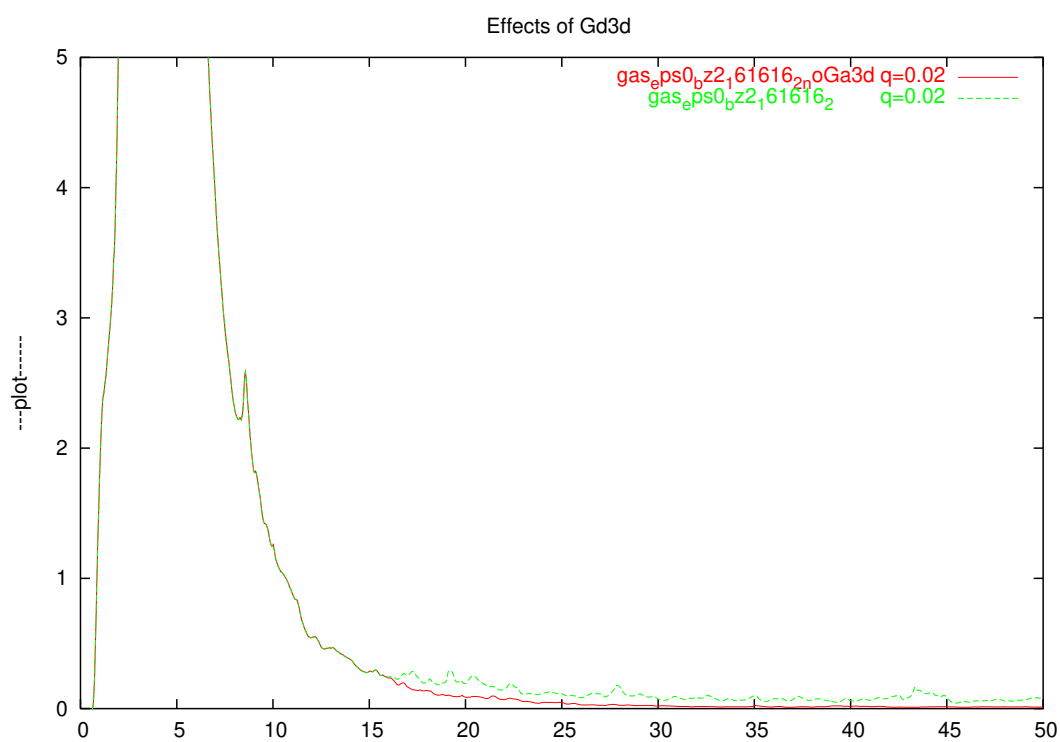


fig008